Milestone 3: Design for Interoperability

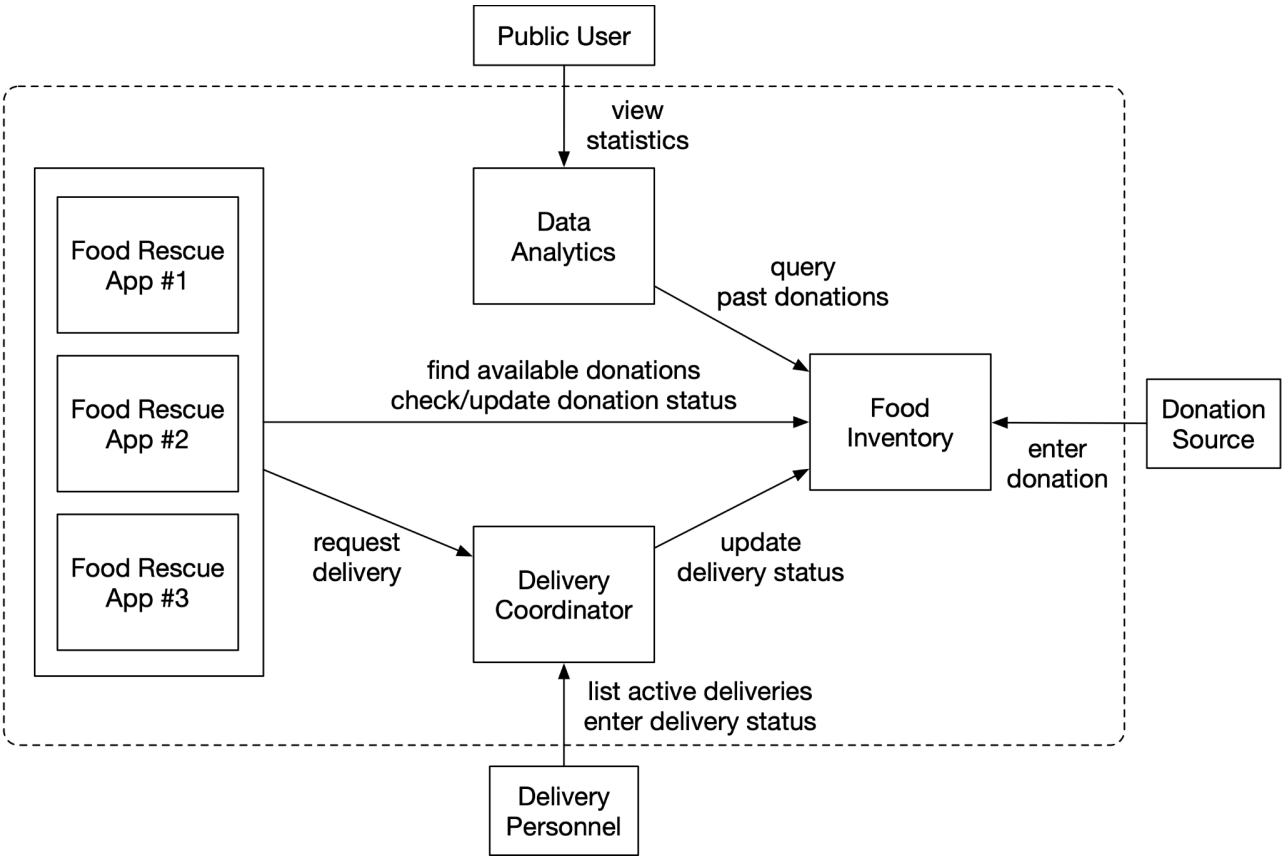
Released: Monday, Mar 2, 2026
Due: Friday, Mar 20, 2026 11:59 pm (on Gradescope)

Learning Objectives

- Extend an existing system to support additional functionality.
- Design software that interoperates with systems developed by other teams.
- Design clear and robust APIs for shared services.
- Practice collaborative system design across multiple teams.

Multi-Service Food Rescue Ecosystem: Introduction

Starting this milestone, each team will be assigned the responsibility of designing (and eventually implementing) a **shared service** that will be deployed and used by all teams. The overall system architecture, including the three shared services and external entities that interact with them, is illustrated in the diagram below:



As shown in the diagram, there will be three shared services (one per team):

- **Food Inventory Service**: Stores the central registry of food donations. Donation sources (e.g., restaurants, grocery stores, cafeterias) enter new donations directly into this service. Individual food rescue apps query the service to find available donations in their area and check or update the status of specific donations (e.g., available, claimed, or picked up). The Delivery Coordinator also calls this service to update a donation's status when a delivery has been completed.
- **Delivery Coordinator Service**: Manages the lifecycle of delivery requests on behalf of food rescue apps. Apps submit a delivery request for a claimed donation, and the Delivery Coordinator tracks the progression of the delivery (e.g., requested, in progress, completed). Delivery personnel interact directly with this service to view a list of active (i.e., unfulfilled) deliveries, self-assign to one, and update its status as the delivery proceeds. For simplicity, we will omit the registration and management of delivery personnel (e.g., onboarding, availability scheduling, or push notifications); you may assume that personnel are already known to the system and will check in on their own to find open requests.
- **Data Analytics Service**: Aggregates data from the Food Inventory to provide community-level statistics accessible to the general public — such as the total amount of food rescued per week, the number of active donations by neighborhood, and trends over a user-specified time period. This service is read-only from the perspective of food rescue apps; its primary consumers are public users and government officials who wish to view statistics about food rescue activity in their community.

Milestone Tasks

In this milestone, you will build on the prototype that you have developed in M2 to support additional features. In addition, you will start working with other teams in the class to design the interface for a new service that will eventually be implemented and used by everyone else.

For this milestone, you will **NOT** need to implement your assigned service; this will be part of the next milestone (**M4**).

Task 1: Prototype Extension

You will extend the existing implementation from M2 to accommodate the following feature:

Delivery request submission and tracking: Once a food recipient has claimed a donation, the physical pickup and delivery of that donation needs to be coordinated. In this task, you will add support for initiating and tracking delivery requests within your application. Specifically, once a recipient has claimed a donation, your system should allow them to submit a request for delivery of that donation. In reality, fulfillment of this request would be managed by an external Delivery Coordinator service (described below in Task 2), which communicates with delivery personnel and updates the Food Inventory Service when the delivery is completed.

For the purpose of this milestone, you should build a **test double component** that stands in for the Delivery Coordinator. The test double should simulate the delivery lifecycle by immediately advancing a request to a hardcoded "in progress" state and returning a simulated confirmation. Once a delivery request has been submitted, the recipient should be able to view its current status from within the application. When the delivery is completed, the donation's status in the Food Inventory should be updated accordingly (via the test double), removing it from the available pool.

Staff testing: Like in M2, the course staff will directly interact with your application to test the new feature. Please ensure the test double is configured so that delivery requests are immediately advanced to a confirmed state when submitted.

Task 2: API Design for Interoperability

This part of the milestone will involve collaboration across multiple teams.

API design activity (Mar 9, 2026): In this task, your team will design (but not yet implement) an API for the service that has been assigned to your team. Since these services will be accessed by multiple applications, **interoperability** is a key quality attribute to be achieved. To facilitate the process of designing APIs together, we will allocate a lecture slot on **Monday, Mar 9, 2026** as a design session. **Every member of your team is expected to be present at this session**.

APIs for external entities: In addition to the API endpoints that will be accessed by the food rescue apps and other services, you should also provide a **public REST-based API endpoint** for the external entity that will be interacting with your service. The functionality of the API endpoint will depend on the particular service that has been assigned to your team:

- **Food Inventory:** The endpoint should allow a Donation Source (e.g., a restaurant or grocery store) to submit a new food donation, including relevant details such as the type of food, quantity, and pickup location.
- **Delivery Coordinator:** The endpoint should allow Delivery Personnel to view a list of active delivery requests and update the status of a delivery they are carrying out (e.g., marking it as completed).
- **Data Analytics:** The endpoint should allow a Public User to view aggregated food rescue statistics, such as the total amount of food rescued over a specified time period or the number of active donations by neighborhood.

API documentation: Your team will document the outcome of this activity as an API documentation that describes (1) the list of interface functions, (2) input and output parameters that are associated with each function, and (3) a contract that describes the pre- and post-condition of each function. In addition, your documentation should include an ontology that describes the meaning of data elements that are passed as parameters, as well as the relationships between those elements.

Tips for Cross-team Collaboration

To ensure seamless integration of your service API with the food rescue apps and other services developed by other teams, **cross-team communication will be crucial.** You should use Slack for cross-team communication and maintain a shared Google Docs document outlining all APIs and assumptions. We recommend that each team designate one member as an "**interface person"** responsible for answering questions from other teams and staying up to date with changes to the APIs of the other shared services. Include the interface person role in your team contract.

Note that the three shared services are not independent: the Delivery Coordinator depends on the Food Inventory to update donation statuses, and the Data Analytics depends on the Food Inventory Service to query aggregate data. Teams responsible for these services should coordinate closely with the Food Inventory team to ensure their APIs are compatible. Remember that other teams will need to write test double components for your API in this milestone. Please finalize your API design and documentation early so that other teams have adequate time to design their systems around it.

Task 3: Design Reflections

In the process of extending your prototype with the new feature, it is likely that the design of your application will undergo some changes. Depending on how you designed your system in M1 & M2, these changes may be straightforward to make (e.g., by adding new components with no or minor changes to the existing ones) or disruptive (i.e., involves significantly changing the existing components).

For this task, prepare a report that reflects on how the design of your system has evolved to accommodate new features and possible future changes. The report should:

1. Describe the changes to the design of your system from M2. Include an updated component diagram that highlights those changes.
2. Describe design decisions that you have made for the API of the shared service in Task 2 and justify how those decisions support service interoperability.

Deliverables

Submit a report as a single PDF file to Gradescope that covers the following items in clearly labeled sections (ideally, each section should start on a new page). **Please correctly map the pages in the PDF to the corresponding sections.**

1. **Deployment (1 paragraph)**: Include the URL for accessing the main frontend of your application.
2. **Testing report (1 pg max)**: A description of how you used a test double to implement and test the delivery request feature, including links to the relevant parts of your GitHub repository (i.e., the test double implementation and the test cases that exercise the new feature).

3. **Shared service API documentation (2 pg max**): A documentation of the API for the shared service that has been assigned to your team, based on Task 2.
4. **Design reflection report** (**2 pg max**): A document reflecting on the (re)design decisions that you have made, as described above in Task 3.
5. **Team contract (1 pg max)**: A documentation of (i) the division of development tasks among team members (including the interface person) and (ii) intermediate milestones, each with a target date and goals achieved.

Grading

This assignment is out of **110** points. For full points, we expect:

- **(30 pt)** A functional web application that extends the prototype from M2 with the new feature.
- (**20 pt**) A documentation of your approach to implementing the test doubles for the new feature & corresponding automated tests.
- **(30 pt)** A documentation of an interface for the shared service that has been assigned to your team, including (1) a contract for each function describing its expected behavior and (**20 pt**) and (2) an ontology for data elements that are passed as input and output parameters (**10 pt**).
- **(20 pt)** A design reflection report with (i) a discussion of how the system design from M2 has been changed to support the new feature, with an updated component diagram (**10 pt**) and (ii) a discussion of design decisions for the shared service interface and justifications for those decisions with respect to interoperability (**10 pt**).
- **(10 pt)** A team contract that describes (i) the division of tasks among team members and (ii) a set of intermediate milestones, their target date, and expected goals.
- **(5 pt)** Bonus social points (same as the previous milestones).